

Garbage Collection - Simple Hinglish Summary

Source: <https://javascript.info/garbage-collection>

Main baat

JavaScript me **memory management automatic hota hai** aur hume dikhta nahi. Primitives, objects, functions sab memory lete hain. Jab koi cheez ki zarurat nahi rehti, to engine use kaise pehchanta hai aur saaf kaise karta hai?

1. Reachability (pahunch me hona)

JS me memory management ka main concept **reachability** hai.

"Reachable" values wo hain jo kisi na kisi tarah **accessible ya usable** hain. Ye memory me **guaranteed** rehti hain.

1. Base set (roots): kuch values hamesha reachable hoti hain, jinhe delete nahi kiya ja sakta. Jaise:

- Abhi chal rahe function ke local variables aur parameters
- Nested calls ki chain ke dusre functions ke local variables aur parameters
- Global variables
- (kuch internal cheezein bhi)

Inhe **roots** kehte hain.

2. Baaki koi bhi value tab reachable maani jati hai jab wo **kisi root se reference ya references ki chain** se pahunchi ja sake.

Jaise: global variable me ek object hai, aur us object ki property dusre object ko refer karti hai, to wo dusra object bhi reachable hai. Aur jise wo refer kare wo bhi.

JS engine me ek background process hota hai jise **garbage collector** kehte hain. Ye saare objects ko monitor karta hai aur jo **unreachable** ho jate hain unhe hata deta hai.

2. Ek simple example

```
javascript

// user ke paas object ka reference hai
let user = {
  name: "John"
};
```

Global variable `user`, object `{name: "John"}` (chhote me "John") ko refer karta hai. Property `name` primitive store karti hai, isliye wo object ke andar hi hai.

Agar `user` ki value overwrite kar do, to reference kho jata hai:

```
javascript  
  
user = null;
```

Ab **John unreachable** ho gaya. Use access karne ka koi tarika nahi, koi reference nahi.
Garbage collector data ko hata dega aur memory free kar dega.

3. Do references

`user` ka reference `admin` me copy kiya:

```
javascript  
  
let user = {  
  name: "John"  
};  
  
let admin = user;
```

Ab agar:

```
javascript  
  
user = null;
```

To bhi object `admin` ke through reachable hai, isliye memory me rahega. `admin` ko bhi overwrite karo tab ye hat sakta hai.

4. Interlinked objects (aapas me jude objects)

Ab thoda complex example: ek parivaar.

```
javascript
```

```
function marry(man, woman) {  
  woman.husband = man;  
  man.wife = woman;  
  
  return {  
    father: man,  
    mother: woman  
  }  
}  
  
let family = marry({  
  name: "John"  
}, {  
  name: "Ann"  
});
```

`marry` do objects ko ek dusre ka reference dekar "shaadi" karata hai aur ek naya object return karta hai jisme dono hain.

Abhi **saare objects reachable** hain.

Ab do references hatao:

```
javascript  
  
delete family.father;  
delete family.mother.husband;
```

Sirf ek reference hatana kaafi nahi hai, kyunki tab bhi saare objects reachable rahenge. Lekin dono hata do to **John ke paas koi incoming reference nahi bacha**.

Outgoing references se koi fark nahi padta. Sirf incoming references hi object ko reachable banate hain. Isliye ab John unreachable hai aur uske saare data ke saath memory se hata diya jayega.

5. Unreachable island (ek poora "dweep" ka gayab hona)

Ho sakta hai **jude hue objects ka poora group** unreachable ho jaye aur memory se hata diya jaye.

Upar wale example me:

```
javascript  
  
family = null;
```

Ab John aur Ann abhi bhi ek dusre se jude hain, dono ke paas incoming references hain. Lekin wo **kaafi nahi hai**.

Purana (family) object root se alag ho gaya, uska koi reference nahi bacha, to **poora "island" unreachable** ho jata hai aur hata diya jata hai.

Ye example dikhata hai ki **reachability ka concept kitna zaruri hai**.

6. Internal algorithm: mark-and-sweep

Basic garbage collection algorithm ko "**mark-and-sweep**" kehte hain.

Ye steps regularly hote hain:

1. Garbage collector **roots** ko lekar unhe "**mark**" karta hai (yaad rakhta hai).
2. Phir unse jude saare references ko visit karke **mark** karta hai.
3. Phir marked objects ke references ko visit karke mark karta hai. Har visit kiya object yaad rakha jata hai, taaki dobara visit na ho.
4. Ye tab tak chalta hai jab tak roots se reachable saare references visit na ho jayein.
5. **Marked ke alawa saare objects hata diye jate hain.**

Ise aise socho: roots se ek bahut bada **paint ka balti** ulta diya jaye, jo saare references se beh kar saare reachable objects ko rang de. **Jo rang nahi paaye, wo hata diye jate hain.**

Optimizations

JS engines is process ko tez karne aur code me delay na aane dene ke liye kai optimizations lagate hain:

- **Generational collection:** objects ko do sets me baanta jata hai: "**naye**" aur "**purane**". Aam code me bahut se objects ki umar chhoti hoti hai (aate hain, kaam karte hain, jaldi mar jate hain). Isliye naye objects ko track karke unki memory saaf karna samajhdari hai. Jo lambe time tak bach jate hain wo "purane" ban jate hain aur unhe kam baar check kiya jata hai.
- **Incremental collection:** bahut objects hon to sabko ek saath check karne me delay aa sakta hai. Isliye engine objects ke set ko **kai hisson me baant** deta hai aur unhe ek ek karke saaf karta hai. Ek bade collection ki jagah **kai chhote collections** hote hain.
- **Idle-time collection:** garbage collector sirf tab chalne ki koshish karta hai jab CPU **idle** ho, taaki execution par asar kam pade.

Aur bhi optimizations hain, lekin alag engines alag techniques use karte hain aur engines ke saath cheezein badalti rehti hain, isliye bina zarurat ke "pehle se" gehra padhna aksar worth nahi hota.

Summary

Mukhya baatein:

- **Garbage collection automatic hota hai.** Hum use force ya rok nahi sakte.

- Objects memory me **tab tak rehte hain jab tak wo reachable hain.**
- **Referenced hona aur reachable hona ek jaisa nahi hai.** Aapas me jude objects ka poor group ek saath unreachable ho sakta hai (unreachable island).

Modern engines advanced algorithms use karte hain. Engine ke andar ki gehri knowledge tab kaam aati hai jab low-level optimizations chahiye. Ise language seekhne ke **baad ka agla step** samajhna chahiye.

Quick Summary

Baat	Yaad rakho
Roots	Current function ke variables, global variables
Reachable	Root se reference ki chain se pahunch me
Unreachable	Root se koi raasta nahi → hata diya jayega
Incoming vs outgoing	Sirf incoming references object ko zinda rakhte hain
Algorithm	Mark-and-sweep
Optimizations	Generational, Incremental, Idle-time
Control	Hum GC ko force ya rok nahi sakte
<code>obj = null</code>	Reference hatata hai (baaki references na ho to object hat jayega)